

Tutorial 10

Exercise 1: Matchings

Let (V, E) be an undirected graph. A matching is a subset $M \subseteq E$ of edges such that for all vertices $v \in V$, at most one edge in M is incident to v (i.e., (V, M) does not contain a path containing more than two vertices). A maximum matching is a matching M such that we have $|M| \geq |M'|$ for every matching M' .

Furthermore, we say that (V, E) is bipartite if $V = L \cup R$ such that

- L and R are disjoint ($L \cap R = \emptyset$) and
 - all edges in E go between L and R ($(u, v) \in E$ implies $\{u, v\} \cap L \neq \emptyset$ and $\{u, v\} \cap R \neq \emptyset$).
1. Describe a task that can be solved by modelling it as an maximum matching problem in a bipartite undirected graph. Bonus points if it involves pizza.
 2. Show how to reduce the maximum matching problem in bipartite undirected graphs to the maximum flow problem.
 3. Show how to express the maximum matching problem in bipartite undirected graphs as a linear program.

Solution:

1. You are in charge of matching donated kidneys to patients. Every kidney is modelled by a vertex in L , every patient needing a kidney is modelled by a vertex in R , and an edge (u, v) between a kidney $u \in L$ and a patient $v \in R$ signifies that the kidney u is compatible with patient v (the blood group matches, etc). The graph is obviously bipartite as edges only connect kidneys and patients, but not kidneys with kidneys or patients with patients.

A matching in this graph corresponds to kidney-patient pairs so that every kidney is assigned to at most one patient and every patient receives at most one kidney (both obviously desirable properties). Also obviously, we try to maximize the size of the matching, i.e., try to assign as many kidneys as possible.

After you have modelled and solved the problem (see below how you did that), you order a pizza for dinner.

2. Given a bipartite undirected graph (V, E) with $V = L \cup R$ we construct a flow network such that from a flow we can construct a matching and vice versa.

Intuitively, we add two new states s and t , connect s to all vertices $\ell \in L$ with an edge (s, ℓ) , direct all edges in E to go from L to R and connect all vertices $r \in R$ to t with an edge (r, t) . Furthermore, each edge gets capacity 1.

Then, the inflow of every vertex $\ell \in L$ is at most 1 and the outflow of every vertex $r \in R$ is at most 1.

Now we apply the integrality theorem: if all edges have an integer capacity (like the one we construct), then the network has a maximal flow f such that $f(u, v)$ is an integer for every edge (u, v) . As we only use the capacity 1, this means we can identify a flow with a matching and vice versa.

Let us describe the reduction in more detail: We begin by defining the flow network $(V \cup \{s, t\}, E', c, s, t)$ where s, t are fresh vertices not in V ,

$$E' = \{(s, \ell) \mid \ell \in L\} \cup \{(\ell, r) \in E \mid \ell \in L \text{ and } r \in R\} \cup \{(r, t) \mid r \in R\},$$

and $c(u, v) = 1$ for all $(u, v) \in E'$.

Now, we show that (V, E) has a matching of size k if and only if $(V \cup \{s, t\}, E', c, s, t)$ has a flow of value k . Then, the size of a maximal matching corresponds to the value of a maximal flow.

First, let M be a matching of size k . We define the flow f_M as follows: For each edge $(\ell, r) \in M$, define $f(s, \ell) = f(\ell, r) = f(r, t) = 1$. All other edges have flow 0 under f . As the edges in the matching do not share vertices, this mapping satisfies indeed flow conservation. Finally, the capacity constraints are trivially satisfied, i.e., f is indeed a flow. Furthermore, its value is k .

Now, let f be a maximal flow in $(V \cup \{s, t\}, E', c, s, t)$. Due to the integrality theorem (cf. Theorem 26.10 in CLRS) and the capacity constraints we can assume $f(u, v) \in \{0, 1\}$ for each edge $(u, v) \in E'$.

We define $M = \{(\ell, r) \in E \mid f(\ell, r) = 1\}$ and claim it is a matching whose size is equal to the value of f . The latter claim follows from flow conservation: at most one unit of flow can reach a vertex of $\ell \in L$. Assume it is one. Then, due to the integrality theorem and conservation of flow, that unit must flow to a unique vertex $r \in R$, i.e., the edge has flow $f(\ell, r) = 1$. Hence, there is at most one edge in M that is incident to ℓ . For vertices in r , the outflow is at most one. Hence, the inflow is also at most one, which means at most one unit of flow can reach r , which means that there is at most one edge in M that is incident to r .

Hence, we have shown the claim and the proof is completed.

3. Given a bipartite undirected graph (V, E) with $V = L \cup R$ we construct an LP capturing the matching problem. We use one decision variable $x_{(u,v)}$ for every edge $(u, v) \in E$ expressing whether (u, v) is in the matching ($x_{(u,v)} = 1$) or not ($x_{(u,v)} = 0$). Note that the decision variables range over the reals, but our constraints are such that in every optimal solution the variables can only have the values 0 or 1.

As we want to maximize the size of the matching, we use the objective function

$$\sum_{(u,v) \in E} x_{(u,v)}.$$

Now, we need to express the requirements on a matching, i.e., that for each vertex at most one edge in the matching is incident to that vertex:

$$\sum_{v \in V} x_{(u,v)} \leq 1 \text{ for all } u \in V.$$

Finally, we express that all variables are indeed in $[0, 1]$:

$$x_{(u,v)} \geq 0 \text{ for all } (u, v) \in E$$

and

$$x_{(u,v)} \leq 1 \text{ for all } (u, v) \in E.$$

Exercise 2: Disjoint paths

Let (V, E) be a directed graph with two dedicated vertices $s \neq t \in V$. A path from s to t is a sequence $v_0 \cdots v_n$ of vertices such that $v_0 = s$, $(v_i, v_{i+1}) \in E$ for all $0 \leq i < n$, and $v_n = t$. We say that the path contains the vertices $v_0, v_1, \dots, v_n$ and that it contains the edges $(v_0, v_1), (v_1, v_2), \dots, (v_{n-1}, v_n)$.

- We say that two paths from s to t are vertex disjoint if the only vertices that are contained in both paths are s and t .
- We say that two paths from s to t are edge disjoint if there is no edge that is contained in both paths.

The maximum vertex-disjoint (edge-disjoint) path problem asks, given a directed graph (V, E) and vertices $s \neq t \in V$, to compute the maximal number of vertex disjoint (edge disjoint) paths from s to t .

1. Describe a problem that can be solved by modelling it as an maximum vertex disjoint path problem in a directed graph.

2. Describe a problem that can be solved by modelling it as an maximum edge disjoint path problem in a directed graph.
3. Show how to reduce the maximum vertex disjoint path problem in directed graphs to the maximum flow problem.
4. Show how to express the maximum vertex disjoint path problem in directed graphs as a linear program.
5. Show how to reduce the maximum edge disjoint path problem in directed graphs to the maximum flow problem.
6. Show how to express the maximum edge disjoint path problem in directed graphs as a linear program.

Solution:

1. Model the internet as a graph consisting of vertices (routers) and edges (links between them). You need to have a reliable connection between two routers s and t . By computing the maximal number of vertex disjoint paths gives the maximal number of independent routes from s to t that do not share a router.

Bonus fact: By Menger's theorem, this number is equal to the number of *routers* you need to remove from the internet to disable all routes between s and t .

2. We again consider the internet example. By computing the maximal number of edge disjoint paths gives the maximal number of independent routes from s to t that do not share a link, but may share routers.

Bonus fact: By Menger's theorem, this number is equal to the number of *links* you need to remove from the internet to disable all routes between s and t .

3. Let (V, E) and $s, t \in V$ be an instance of the maximum vertex disjoint path problem in directed graphs.

We define the network (V', E', c, s', t') with

- $V' = V \times \{i, o\}$,
- $E' = \{((v, i), (v, o)) \mid v \in V\} \cup \{((u, o), (v, i)) \mid (u, v) \in E\}$,
- $c(u, v) = 1$ for all $(u, v) \in E'$,
- $s' = (s, o)$, and
- $t' = (t, i)$.

Draw the resulting network for a small graph to get an intuition for the construction before you continue reading.

Intuitively, we split each vertex in two and add an edge with capacity one in between. This ensures that at most one unit of flow goes through each vertex. This ensures that each flow through the network corresponds to a union of vertex-disjoint paths from (s, o) to (t, i) and vice versa.

Thus, the value of a maximal flow is equal to the maximal number of vertex-disjoint paths from s to t in (V, E) . The formal proof of correctness of this construction is very similar to that of Exercise 1 and relies on the integrality theorem.

4. Instead of developing a bespoke LP encoding of the maximum vertex disjoint path problem in directed graphs, we apply reductions: The maximum vertex disjoint path problem in directed graphs can be reduced to the maximum flow problem (as shown above), which in turn can be reduced to LP (as seen in the lecture).

5. Let (V, E) and $s, t \in V$ be an instance of the maximum edge disjoint path problem in directed graphs. We proceed as in Item 3, but do not need to split the vertices.

Instead we define the network (V, E, c, s, t) with capacity $c(u, v) = 1$ for all $(u, v) \in E$. Then, the value of a maximal flow is equal to the maximal number of vertex-disjoint paths from s to t in (V, E) . The formal proof of correctness of this construction is very similar to that of Exercise 1 and relies on the integrality theorem.

6. Instead of developing a bespoke LP encoding of the maximum edge disjoint path problem in directed graphs, we apply reductions: The maximum edge disjoint path problem in directed graphs can be reduced to the maximum flow problem (as shown above), which in turn can be reduced to LP (as seen in the lecture).

Exercise 3: The minimum-cost flow problem

In the minimum-cost flow problem, each edge (u, v) in a flow network comes with a cost-coefficient $a(u, v)$ and a throughput $d \in \mathbb{R}$. One is interested in determining a flow f of value d (if one exists) that minimizes the cost of the flow, which is obtained by summing up the cost of f for each edge (u, v) , defined as $a(u, v) \cdot f(u, v)$.

1. Define the minimum-cost flow problem formally.
2. Show how to express the minimum-cost flow problem as a linear program.

Solution:

1. A flow network with costs $((V, E, c, s, t), a)$ consists of a flow network (V, E, c, s, t) (as defined in the lecture) and a cost function $a: E \rightarrow \mathbb{R}$.

The minimum-cost flow problem asks, given a flow network with costs $((V, E, c, s, t), a)$ and a throughput $d \in \mathbb{R}$, to compute a flow f in (V, E, c, s, t) of value d that minimizes the cost

$$\sum_{(u,v) \in E} a(u, v) \cdot f(u, v).$$

2. We adapt the LP formulation presented in the lecture. In particular, we replace the original objective function (which aimed to maximize the flow) by an objective function aiming to minimize the cost of the flow and add a constraint expressing that the value of the flow is d .

As before, we use decision variables $x_{(u,v)}$ for each $(u, v) \in E$, representing the flow through edge (u, v) .

As said above, we are interested in minimizing the cost of the flow, so we use the objective function

$$\sum_{(u,v) \in E} a(u, v) \cdot x_{(u,v)}.$$

The first four constraints are as in the lecture, expressing the capacity constraint and the flow constraint.

- (a) $x_{(u,v)} \geq 0$ for all $(u, v) \in E$ (lower bound of capacity constraint)
- (b) $x_{(u,v)} \leq c(u, v)$ for all $(u, v) \in E$ (upper bound of capacity constraint)
- (c) $\sum_{(u,v) \in E} f(u, v) - \sum_{(v,u) \in E} f(v, u) \leq 0$ for all $v \in V \setminus \{s, t\}$ (one “half” of flow constraint)
- (d) $\sum_{(v,u) \in E} f(v, u) - \sum_{(u,v) \in E} f(u, v) \leq 0$ for all $v \in V \setminus \{s, t\}$ (other “half” of flow constraint)

The new constraints we add express that the value of the flow is exactly d (compare them to the original objective function):

$$\sum_{(s,v) \in E} x_{(s,v)} - \sum_{(v,s) \in E} x_{(v,s)} \leq d$$

and

$$\sum_{(v,s) \in E} x_{(v,s)} - \sum_{(s,v) \in E} x_{(s,v)} \leq -d$$

are equivalent to $\sum_{(s,v) \in E} x_{(s,v)} - \sum_{(v,s) \in E} x_{(v,s)} = d$, which expresses that the value of the flow is d .